

Seek Wander

As-Built Architecture Review

Document Version	2.0 — Phase 11 Complete
Date	March 2026
Status	[STATUS: PRODUCTION READY] All 11 phases deployed
Production URL	travel-planner-v2-pearl.vercel.app
Repository	github.com/RomaanR/travel-planner-v2
Classification	Confidential — Technical Review

This document is a complete technical review of Seek Wander's production architecture. All systems described are live, committed, and deployed. Coverage: system architecture, security controls, data schemas, cost model, affiliate integration, PWA/offline strategy, and deployment topology. No fluff — every section is sourced directly from the production codebase.

Product Summary [STATUS: PRODUCTION READY]

Seek Wander is a luxury AI travel concierge that generates bespoke, multi-day itineraries. Users complete a 7-field intake form; the system returns a geographically-verified, chronologically-ordered day plan with restaurant recommendations, hidden gems, transit estimates, curated hotel stays, and destination photography — in 15–20 seconds.

! ZERO-HALLUCINATION GUARANTEE

Every AI-generated location is enriched post-generation via **Google Places API**, so ratings, opening hours, and photos are ground-truthed before the user ever sees the output. The model cannot fabricate a location that passes Place enrichment — non-existent places return null and render gracefully without crashing.

System Architecture

Layer Overview

Client	Next.js 14 App Router, Tailwind CSS , Framer Motion 12 , @react-google-maps/api	Rendering, animation, form intake, map display
Edge/Serverless	Vercel — Next.js API Routes	Request handling, AI orchestration, enrichment pipeline
AI Engine	Anthropic claude-sonnet-4-6 (max_tokens: 8192)	Itinerary generation from structured user profile
Place Data	Google Places API (Text Search + Details)	Location verification, photos, ratings, opening hours
Auth	Clerk v6	User identity, session management, modal sign-in
Primary DB	Supabase PostgreSQL via Prisma 7	Trip storage, PlaceCache, CostLog
Rate Limiting	Upstash Redis	Sliding window per userId/IP before Anthropic call
PWA	@serwist/next	Service worker, photo CacheFirst, app shell offline
CDN/Hosting	Vercel Pro	Global edge delivery, serverless functions, cron jobs

Core Request Pipeline — POST /api/itinerary

1. Zod Validation	ItinerarySchema.safeParse() — 12 fields	400 + fieldErrors
2. Rate Limit Check	Upstash slidingWindow(5, '1 h') per userId/IP	429 + X-RateLimit-* headers
3. AI Generation	claude-sonnet-4-6 with injection-resistant SYSTEM_PROMPT	Phase 4 JSON repair
4. JSON Sanitisation	sanitizeJson() regex + fallback Anthropic repair call	500 (both phases fail)
5. Place Enrichment	enrichPlace() × N — PlaceCache first, then Google Places	Graceful null (item still renders)
6. Transit Calc	Haversine formula — local math, zero API calls	Never fails
7. Cost Logging	CostLog.create() awaited before Response.json()	Non-fatal try/catch

Technology Stack [STATUS: ALL DEPENDENCIES LOCKED]

Framework	Next.js App Router	14	TypeScript; server + client components; force-dynamic where needed
Styling	Tailwind CSS	v3	Custom design tokens; rounded-none enforced globally
Animation	Framer Motion	12	AnimatePresence throughout; print:hidden on all motion wrappers
Maps	@react-google-maps/api	Latest	Places Autocomplete, GoogleMap, custom SVG Markers, Polyline
AI	@anthropic-ai/sdk	^0.78	claude-sonnet-4-6; max_tokens 8192; stream disabled
Auth	@clerk/nextjs	v6 ONLY	DO NOT UPGRADE TO v7 — v7 requires Next.js 15; is v7-only and must not be used
ORM	Prisma	7	Supabase PostgreSQL; PgBouncer port 6543 runtime; port 5432 for migrations
Rate Limiting	@upstash/ratelimit	Latest	Redis-backed; slidingWindow; key = Clerk userId (auth) or IP (unauth)
Notifications	sonner	2	Branded toast layer; richColors; id-based mutation (no stacking)
PWA	@servist/next	Latest	sw.ts → public/sw.js; disabled in development
Validation	zod	4	All POST bodies; z.infer<> is the source-of-truth type

! CRITICAL VERSION LOCK — @clerk/nextjs

@clerk/nextjs MUST stay at v6. Clerk v7 requires Next.js 15 and introduces breaking API changes: <Show> replaces <SignedIn>/<SignedOut>; auth() return type changes; middleware API differs. Any dependency update that pulls in @clerk/nextjs@7+ will silently break the entire auth layer without a compile error.

Security Architecture [STATUS: 10 CONTROLS ACTIVE]

! DEFENCE-IN-DEPTH MODEL

Each control layer assumes the previous has already been bypassed. **Zod** fires before any computation. **Upstash** fires before any Anthropic token is consumed. **SYSTEM_PROMPT injection defence** is independent of Zod. IDOR checks are always post-fetch — never encoded in the WHERE clause alone.

Zod ItinerarySchema safeParse() — 12 fields	Malformed payloads crashing the AI pipeline or triggering DB exceptions	Clean 400s with per-field feedback; zero 500s from bad user input
SYSTEM_PROMPT injection defence (system: param, not messages[])	Adversarial destination / interest fields hijacking AI role or exfiltrating internal prompts	Itinerary quality guaranteed; system param is processed at a higher trust tier
Upstash Redis rate limit slidingWindow(5, '1 h') per userId / IP	API cost exhaustion: single actor flooding Anthropic spend to exhaust billing limit	Hard cost ceiling per user; 429 returned before any AI token consumed
AI JSON Self-Healing (sanitizeJson + JSON_REPAIR_PROMPT)	Malformed Claude output (markdown fences, trailing commas) reaching the UI as a broken or blank itinerary	0% broken itinerary displays; silent two-phase repair invisible to users
Google API Key Split (NEXT_PUBLIC_client / MAPS_SERVER_KEY server)	Client key theft enabling unrestricted Google Maps API usage billed to the project	Client key: HTTP referrer restricted to Vercel + localhost. Server key: never transmitted to browser
IDOR Prevention (post-fetch userId ownership check on /trips/[id])	User A accessing User B's saved trips via direct ID in URL	Data privacy compliance; neutral notFound() leaks zero ownership information
Auth Guard on GET /api/trips (Clerk userId required)	Unauthenticated scraping of all saved user trips	All queries scoped to { where: { userId } }; cross-user access structurally impossible
HTTP Security Headers (X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy)	Clickjacking, MIME-type confusion, cross-origin data leaks	Passes OWASP header checklist; applied to all routes via next.config.mjs headers()
Admin Neutral 404 on /admin/metrics for non-ADMIN_USER_ID	Route enumeration revealing admin panel existence to non-admin users	notFound() (not redirect) — the route's existence is not disclosed

Cron Bearer Auth (Authorization: Bearer CRON_SECRET)	External actors triggering daily PlaceCache cleanup on demand	Database integrity; Vercel injects CRON_SECRET on every scheduled invocation

Data Architecture

Prisma Schema

Model: Trip

id	String UUID	Primary key
userId	String	Clerk userId — no FK constraint
destination	String	Human-readable destination name
days	Int	Trip duration in days
itineraryData	Json	Full ItineraryResponse blob — cast via trip.itineraryData as unknown as ItineraryResponse
createdAt	DateTime	Auto-set on create

Model: PlaceCache

id	String UUID	Primary key
cacheKey	String UNIQUE	'{normalizedName}{normalizedCity}' — toLowerCase + trim on both parts
photoUrl	String?	Google Places photo URL (photoreference param — NOT photo_reference)
rating	Float?	Google Places rating
userRatingsTotal	Int?	Review count
hoursOpen	String?	'9:00 AM – 9:00 PM' — today's hours, day prefix stripped. Used by parseOpenNow().
priceLevel	Int?	0–4 price scale
updatedAt	DateTime	@updatedAt — Prisma auto-refreshes on every upsert. Used by Vercel Cron GC (14-day cutoff).

Model: CostLog

userId	String?	Clerk userId — null if unauthenticated
destination	String	Trip destination

aiCost	Decimal(10,6)	Anthropic spend in USD
googleCost	Decimal(10,6)	Google Places spend in USD
totalCost	Decimal(10,6)	aiCost + googleCost
cacheHits	Int	PlaceCache hits — zero Google cost
cacheMisses	Int	Fresh Google API calls made
createdAt	DateTime	Auto-set on create

PlaceCache — Efficiency Model

Cache key format	{normalize(name)}{normalize(city)} — toLowerCase + trim on both parts ensures consistent hits
Cache hit path	Zero Google API calls. parseOpenNow(hoursOpen, lng) computes live open/closed status locally using Math.round(lng / 15) hours as UTC offset (±30 min accuracy — sufficient for a planning app).
Cache miss path	Google Places Text Search (\$0.032) + Place Details (\$0.017, skipped for NATURE/ADVENTURE categories). Result upserted to PlaceCache (fire-and-forget, non-fatal).
Savings per item	\$0.049 (Text Search + Details). At 60% cache hit rate: ~\$0.03 saved per generation on enrichment alone.
Garbage collection	Vercel Cron runs GET /api/cron/cleanup daily at 00:00 UTC. Deletes rows where updatedAt < 14 days ago. @updatedAt auto-refreshes on every cache hit upsert — frequently accessed places naturally survive.

Observability & Cost Control [STATUS: LIVE — /admin/metrics]

/admin/metrics — Internal BI Dashboard

Accessible only to the **ADMIN_USER_ID** Clerk userId (env var, **.trim()**'d to prevent Vercel trailing-newline bug). Non-admin access returns **notFound()** — route existence not leaked. Shows last 200 CostLog rows. Cost colour-coding: green < \$0.15, amber > \$0.50.

Total Spent	SUM(totalCost) across all CostLog rows
Total Generations	COUNT(CostLog)
Avg Cost / Trip	totalSpent / totalGenerations
Cache Hit Rate	SUM(cacheHits) / SUM(cacheHits + cacheMisses) — target > 60%
Cost by Provider	SUM(aiCost) vs SUM(googleCost) — Claude vs Google split per generation

Unit Economics Reference

Anthropic input tokens	\$3.00 / 1M tokens
Anthropic output tokens	\$15.00 / 1M tokens
Google Text Search	\$0.032 / call
Google Place Details	\$0.017 / call
Cache cold generation	\$0.08 – \$0.14
Cache warm generation (60% hits)	\$0.05 – \$0.09

Affiliate Monetization Architecture [STATUS: LIVE — AID 4013143]

Booking.com Integration — Technical Specification

Affiliate ID	4013143 — Seek Wander partner account. No SDK — URL-based integration only.
Utility: createAffiliateUrl()	src/lib/affiliate.ts. Signature: createAffiliateUrl(hotelName: string, destination: string): string. Output: https://www.booking.com/searchresults.html?ss={encodeURIComponent(hotelName + ' ' + destination)}&aid;=4013143
StayCard component	src/components/StayCard.tsx. Renders as motion.a (whileHover: y:-2). ArrowUpRight icon + 'View on Booking.com' micro-copy. target='_blank' rel='noopener noreferrer'. print:hidden — excluded from PDF dossier.
Data shape	recommendedStays[]: Array<{ name: string; description: string; neighbourhood: string }>. Generated by claude-sonnet-4-6 when accommodationStatus === 'needed'. Absent when accommodationStatus === 'booked'.
Placement in ItineraryViewer	Rendered after the daily timeline section, before the bottomSection slot. Conditional: only shown when itinerary.recommendedStays?.length > 0.
Attribution	Booking.com 30-day last-click cookie. Commission: ~25–35% of Booking.com margin per completed booking.
Analytics	Booking.com partner dashboard tracks impressions, clicks, and completed bookings under AID 4013143.

NOTE — ACCOMMODATION BRANCHING IN SYSTEM_PROMPT

When `accommodationStatus === 'needed'`: the `SYSTEM_PROMPT` instructs **claude-sonnet-4-6** to generate `recommendedStays[]` (3 curated hotels with name, description, neighbourhood). When `accommodationStatus === 'booked'`: the user-supplied `hotelName` is woven into the itinerary narrative as the base hotel and `recommendedStays[]` is omitted entirely. This prevents affiliate cards appearing when the user has already committed — protecting UX trust while maximising affiliate click opportunity on undecided users.

Monetization Flow Diagram

1	User	Sets <code>accommodationStatus = 'needed'</code> in <code>CurationForm</code>
2	API Route	<code>SYSTEM_PROMPT</code> branch: instructs Claude to generate <code>recommendedStays[]</code>
3	Claude	Returns <code>recommendedStays[]</code> with 3 hotel objects in JSON response

4	ItineraryViewer	StayCard renders per hotel; createAffiliateUrl() called client-side
5	User	Clicks StayCard → opens Booking.com with AID 4013143 in URL
6	Booking.com	30-day cookie set; commission triggered on confirmed booking
7	/admin/metrics	Revenue tracked via Booking.com partner dashboard (external)

PWA & Offline Architecture [STATUS: INSTALLABLE PWA]

Service Worker Strategy — @serwist/next

Source: `src/app/sw.ts` → compiled to `public/sw.js` by the `@serwist/next` webpack plugin (configured in `next.config.mjs` via `withSerwist()`). Service worker is **disabled** when `process.env.NODE_ENV === 'development'` to prevent stale cache contaminating hot-reload sessions.

CacheFirst	Google Places photo URLs (contain photoreference parameter)	30-day TTL, 100-entry cap	Photos never change once fetched. CacheFirst = zero network request on repeat views. Cap prevents unbounded storage growth.
StaleWhileRevalidate (defaultCache)	Next.js JS chunks, CSS bundles, static fonts	Versioned by build hash	App shell loads from cache instantly. New deploy invalidates stale chunks via Next.js build hash in URL.
NetworkOnly (implicit)	API routes (<code>/api/*</code>), Clerk auth endpoints, Supabase queries	—	Dynamic data must always be fresh. Auth tokens must not be served from cache.

localStorage Vault — Offline Trip Archive

The `useOfflineTrips` hook implements a two-phase offline detection + recovery strategy. The vault key is `seek_wander_archive` in `localStorage`.

Phase 1 — Pre-fetch check	<code>navigator.onLine === false</code> on component mount	Load from <code>localStorage</code> immediately. No network request attempted. <code>isOffline = true</code> . Offline banner shown.
Phase 2 — Fetch-then-cache	<code>navigator.onLine === true</code>	Fetch <code>GET /api/trips</code> (Clerk-authed). Success: persist to <code>localStorage</code> , render live data. Failure: load from <code>localStorage</code> , <code>isOffline = true</code> .
Phase 3 — Cache persistence	Every successful <code>/api/trips</code> response	Serialize <code>CachedTrip[]</code> to <code>localStorage</code> . Overwrites any stale data. Next offline visit uses this fresh snapshot.
Vault key	<code>seek_wander_archive</code>	
Stored shape	<code>CachedTrip[]</code> — <code>{id, userId, destination, days, createdAt, itineraryData, photoUrl}</code> . <code>photoUrl</code> is pre-resolved server-side via <code>MAPS_SERVER_KEY</code> .	
Photo URL resolution	Computed in <code>GET /api/trips</code> route handler using <code>MAPS_SERVER_KEY</code> . Client (<code>TripsClient.tsx</code>) receives plain URL strings — never holds the server API key.	

Offline indicator	bg-ink text-paper banner with WifiOff icon. Shown whenever <code>isOffline === true</code> , regardless of <code>navigator.onLine</code> (covers server-down scenarios too).
-------------------	--

! IMPORTANT — WHY TWO-PHASE?

Phase 1 (`navigator.onLine` check) catches true offline scenarios instantly without making a failing network request. Phase 2 (fetch failure fallback) catches scenarios where the device reports online but the server is down or the Clerk token has expired. Combined, the vault degrades gracefully under all real-world conditions.

PWA Manifest

Source	<code>src/app/manifest.ts</code> — Next.js <code>MetadataRoute.Manifest</code> type
App name	Seek Wander (no 'AI' in name)
Display mode	standalone — hides browser chrome on install
Background color	<code>#F6F1EB</code> — matches paper palette on splash screen
Theme color	<code>#1B1817</code> — dark ink for status bar
Icons	<code>/icon-192x192.png</code> and <code>/icon-512x512.png</code> in <code>/public/</code>
iOS meta	<code>appleWebApp: capable:true, statusBarStyle:default, title:'Seek Wander'</code>

Mobile-First Design

Responsive Layout Grid

Mobile (< 768px)	Full-width single-column timeline	h-52 banner above timeline (collapses on scroll)
Desktop (≥ 768px)	55% editorial timeline / 45% sticky map split-screen	Sticky right panel — position:sticky, h-full
Print (@media print)	Single-column, all days sequential, page-break between days	print:hidden — entirely removed from output

MobileMenu Overlay — CSS Stacking Context Resolution

! ARCHITECTURAL LESSON LEARNED

Root cause of the mobile menu transparency bug: `backdrop-blur-sm` applied to the root `motion.nav` element in `Navbar.tsx` creates a new CSS stacking context. Any fixed-positioned descendant is anchored to that ancestor — not the viewport. The overlay's `fixed inset-0` was therefore anchored to the ~64px Navbar bar, not the full screen. `background-color`, `z-index`, and `opacity` were all irrelevant — the overlay literally did not extend past the Navbar's boundaries. **Resolution:** `createPortal(overlay, document.body)` moves the DOM node outside the Navbar tree entirely. `fixed inset-0` then covers the true viewport. SSR guard: `mounted` state + `useEffect` ensures `document.body` is available before the portal renders. **Architectural rule:** Any fixed full-screen overlay rendered inside a component with `backdrop-filter`, `transform`, `filter`, `will-change`, or `perspective` MUST use `createPortal(..., document.body)`.

High-Contrast Mobile UI Rules

Hamburger button	<code>bg-paper border border-ink/20 rounded-full shadow-md p-3</code> — solid pill ensures visibility against dark photographic hero backgrounds
Overlay background	<code>bg-paper (#F5F0E8)</code> + inline style <code>backgroundColor</code> fallback — dual approach guarantees solid background independent of Tailwind JIT
Scroll lock	<code>document.body.style.overflow = 'hidden'</code> on open, <code>'unset'</code> on close — <code>useEffect</code> cleanup reverts on unmount
Portal SSR guard	<code>const [mounted, setMounted] = useState(false) + useEffect(() => { setMounted(true); }, [])</code> — portal only renders after hydration; <code>document.body</code> is unavailable on server

Deployment Topology [STATUS: PRODUCTION ON VERCEL]

Web App	Vercel Pro	Auto-deploy on main branch push. postinstall: 'prisma generate' regenerates Prisma client with correct Amazon Linux 2023 binary.
Database	Supabase PostgreSQL	DATABASE_URL: PgBouncer port 6543 (runtime queries). DIRECT_URL: port 5432 (Prisma migrations + db push). Both required in .env.local.
Auth	Clerk v6	Modal sign-in — no dedicated auth pages. Conditional ClerkProvider (skipped if key absent).
Rate Limiting	Upstash Redis	REST API — stateless, no persistent connection. @upstash/ratelimit slidingWindow(5, '1 h').
AI	Anthropic API	claude-sonnet-4-6, server-side only. ANTHROPIC_API_KEY never transmitted to client.
Maps	Google Cloud	Two keys: NEXT_PUBLIC_ (referrer-restricted, browser) + MAPS_SERVER_KEY (Places API only, no referrer, server).
Cron	Vercel Cron	vercel.json: '0 0 * * *' — daily PlaceCache GC at midnight UTC.
Affiliate	Booking.com	AID 4013143. No SDK — URL-based. Zero infrastructure cost.

Environment Variables

ANTHROPIC_API_KEY	POST /api/itinerary	Server only — never in NEXT_PUBLIC_
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY	Browser — Maps JS, Autocomplete	HTTP referrer restricted: Vercel + localhost
MAPS_SERVER_KEY	route.ts enrichment, getPlacePhoto.ts	Places API only; no referrer restriction
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY	ClerkProvider in layout.tsx	Public (safe in browser)
CLERK_SECRET_KEY	Clerk middleware + auth()	Server only
DATABASE_URL	Prisma runtime queries	PgBouncer port 6543
DIRECT_URL	Prisma migrations / db push	Direct port 5432

ADMIN_USER_ID	/admin/metrics auth gate	Server only; must be .trim()'d
CRON_SECRET	/api/cron/cleanup bearer check	Server only; generate with openssl rand -hex 32
UPSTASH_REDIS_REST_URL	ratelimit.ts — Upstash client	Server only
UPSTASH_REDIS_REST_TOKEN	ratelimit.ts — Upstash client	Server only

! PRISMA DEPLOYMENT NOTE

package.json includes **postinstall: 'prisma generate'** so Vercel regenerates the Prisma client with the correct Amazon Linux 2023 binary after npm install. Do NOT pin **binaryTargets** in schema.prisma — Prisma auto-detects the correct platform binary. Pinning to **rhel-openssl-1.0.x** will break on Vercel (Amazon Linux 2023 uses OpenSSL 3.x).

Phase Completion Audit

1 — Core Engine	Next.js + Claude + Google Maps baseline	[COM PLET E]
2 — Concierge UX	7-field form, split-screen, enrichment, Haversine transit	[COM PLET E]
3 — Ultra-Luxury UI	Tabbed nav, cost badges, transit connectors, day-centric markers	[COM PLET E]
4 — Auth	Clerk v6, conditional ClerkProvider, NavbarAuth, custom 404	[COM PLET E]
5 — Persistence	Prisma + Supabase, saveTrip, /trips, /trips/[id], IDOR enforcement	[COM PLET E]
6 — PWA & Sharing	@serwist/next, /shared/[id] OG, ShareButton, PDF export	[COM PLET E]
7 — Timeline Refactor	timeline: TimelineItem[], normalizeDayPlan() shim	[COM PLET E]
8 — Margin Protection	PlaceCache, Haversine, CostLog, /admin/metrics, Cron, Zod, HTTP headers	[COM PLET E]
9 — Affiliate	Booking.com AID 4013143, StayCard, createAffiliateUrl(), accommodation branching	[COM PLET E]
10 — UX & Polish	GenerationLoader, Sonner toasts, UnauthenticatedState, EmptyTripsState, MobileMenu portal fix	[COM PLET E]
11 — Mobile & Resilience	useOfflineTrips (seek_wander_archive), /api/trips, Upstash rate limiting, AI JSON self-healing, dynamic OG metadata	[COM PLET E]
12 — Stripe Paywall	\$4.99 / generation after 1 free — Stripe Checkout, usage tracking	[NEX T]